

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 6

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement :

Design and implement a Convolutional Neural Network (CNN) for image classification

Algorithm :

1. Load the butterfly dataset containing 75 classes of butterfly images.
2. Read image filenames and corresponding labels from Training_set.csv.
3. Split the labeled data into 80% training and 20% validation sets using stratified sampling.
4. Resize images to $224 \times 224 \times 3$ and normalize pixel values to $[0,1]$.
5. Convert butterfly class labels into numerical labels.
6. Build a custom CNN consisting of:
 - Three convolutional layers with 32, 64, and 128 filters
 - ReLU activation
 - Max-pooling after each convolutional layer
 - Flatten layer
 - Dense layer with 128 neurons
 - Dropout layer
 - Output layer with 75 neurons and Softmax

7. Compile the CNN using the Adam optimizer and Sparse Categorical Cross-Entropy loss.
8. Train the model on the training set for 10 epochs.
9. Evaluate the trained model on the validation set using accuracy and loss.
10. Generate predictions and evaluate performance using a classification report and confusion matrix.

Code & Output :

```
!pip install -q kaggle

import os
os.environ["KAGGLE_API_TOKEN"] = "KGAT_f9c7ca54c817498323213beac2565163"

!kaggle datasets list -s butterfly

.. Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)
ref title size lastUpdated downloadCount voteCou
-----
phuchthaiv02/butterfly-image-classification Butterfly Image Classification 236814249 2026-06-26 08:58:37.513000 31900 2
veeralakrishna/butterfly-dataset Butterfly Dataset 476468534 2020-01-10 12:07:42.413000 5543 1
gpiosenka/butterfly-images40-species Butterfly & Moths Image Classification 100 species 418471259 2023-04-21 22:53:16.223000 16365 1
alessiocorrado99/animals10 Animals-10 614087302 2019-12-12 20:46:33.797000 122026 12
pnkjgpt/butterfly-classification-dataset Butterfly Classification Dataset 447173225 2021-06-30 06:03:06.297000 995
thedevasator/smithsonian-butterfly-dataset Smithsonian Butterfly Dataset 327610413 2023-12-06 01:36:19.803000 375
ayshine/butterfly Butterfly 9566789484 2018-10-28 01:02:42.853000 364
antoreepjana/animals-detection-images-dataset Animals Detection Images Dataset 9575388685 2021-05-05 18:07:46.973000 48573 2
datasciencedonut/olympic-swimming-1912-to-2020 Olympic Swimming History (1912 to 2020) 77664 2023-04-18 01:09:15.173000 4109
razimw/bird-vs-butterfly BIRD VS BUTTERFLY 433266760 2020-10-31 01:06:46.300000 378
utkarshsaxenadn/attention-unet-butterfly-segmentation Attention Unet Butterfly Segmentation 125652338 2022-09-29 08:56:05.537000 199
mefionmefalchkt01/butterfly-detection-dalabin Calkchkt01 Butterfly Detection @ Dalabin 3060333 2022-06-17 17:40:33.000000 604

!kaggle datasets download -d phuchthaiv02/butterfly-image-classification

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)
Dataset URL: https://www.kaggle.com/datasets/phuchthaiv02/butterfly-image-classification
License(s): CC0-1.0
Downloading butterfly-image-classification.zip to /content
100% 226M/226M [00:02<00:00, 88.3MB/s]

!unzip -q butterfly-image-classification.zip -d butterfly_dataset

import os

dataset_path = "/content/butterfly_dataset"

os.listdir(dataset_path)

... ['Testing_set.csv', 'train', 'test', 'Training_set.csv']
```

```

import pandas as pd

train_csv = pd.read_csv(
    "/content/butterfly_dataset/Training_set.csv"
)

train_csv.head()

```

```

...
  filename      label
0  Image_1.jpg  SOUTHERN DOGFACE
1  Image_2.jpg      ADONIS
2  Image_3.jpg  BROWN SIPROETA
3  Image_4.jpg    MONARCH
4  Image_5.jpg  GREEN CELLED CATTLEHEART

```

Next steps: [Generate code with train_csv](#) [New interactive sheet](#)

```
train_csv.shape
```

```
(6499, 2)
```

```
train_csv.columns
```

```
Index(['filename', 'label'], dtype='object')
```

```
num_classes = train_csv['label'].nunique()
```

```
print("Number of classes:", num_classes)
```

```
Number of classes: 75
```

```

class_counts = train_csv['label'].value_counts()

print(class_counts)

```

```

...
label
MOURNING CLOAK    131
SLEEPY ORANGE    107
ATALA             100
BROWN SIPROETA    99
SCARCE SWALLOW   97
...
AMERICAN SNOOT    74
GOLD BANDED       73
MALACHITE         73
CRIMSON PATCH     72
WOOD SATYR        71
Name: count, Length: 75, dtype: int64

```

```

print("Minimum images in a class:", class_counts.min())
print("Maximum images in a class:", class_counts.max())

```

```

Minimum images in a class: 71
Maximum images in a class: 131

```

```

import matplotlib.pyplot as plt
from PIL import Image
import os

fig, axes = plt.subplots(2, 5, figsize=(15, 7))

for i, ax in enumerate(axes.flat):
    row = train_csv.iloc[i]

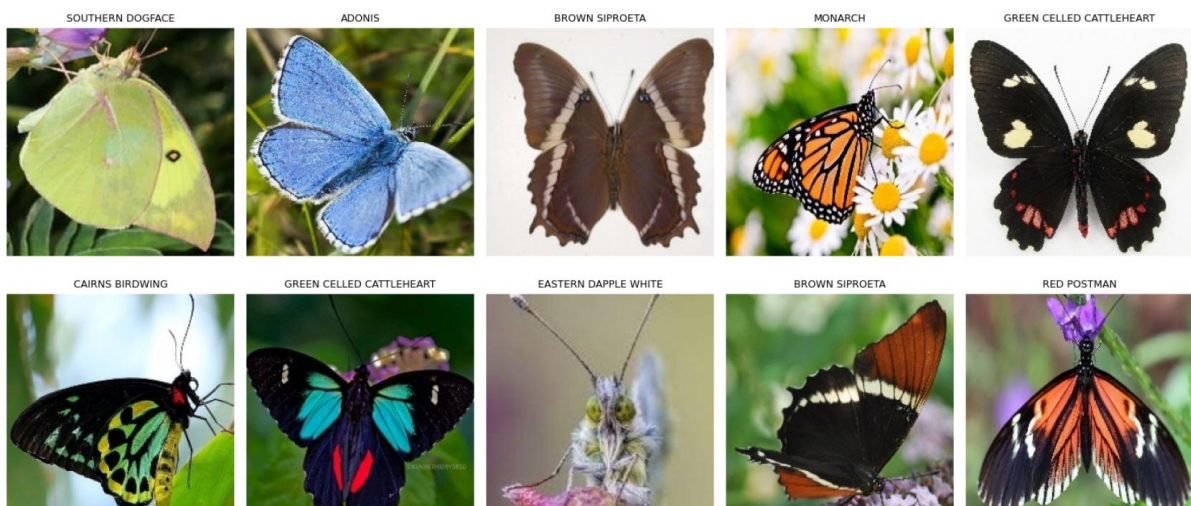
    img_path = os.path.join(
        "/content/butterfly_dataset/train",
        row["filename"]
    )

    img = Image.open(img_path)

    ax.imshow(img)
    ax.set_title(row["label"], fontsize=9)
    ax.axis("off")

plt.tight_layout()
plt.show()

```



```
I
S
img = Image.open(
    "/content/butterfly_dataset/train/" + train_csv.iloc[0]["filename"]
)
```

```
print("Original image size:", img.size)
print("Image mode:", img.mode)
```

```
Original image size: (224, 224)
Image mode: RGB
```

```
I
S
from sklearn.model_selection import train_test_split

train_df, val_df = train_test_split(
    train_csv,
    test_size=0.20,
    stratify=train_csv["label"],
    random_state=42
)
```

```
print("Training images:", len(train_df))
print("Validation images:", len(val_df))
```

```
Training images: 5199
Validation images: 1300
```

```
print("Training classes:", train_df["label"].nunique())
print("Validation classes:", val_df["label"].nunique())
```

```
Training classes: 75
Validation classes: 75
```

```
import tensorflow as tf

print("TensorFlow version:", tf.__version__)
```

```
TensorFlow version: 2.20.0
```

```
print("GPU available:", tf.config.list_physical_devices('GPU'))
```

```
GPU available: []
```

```
▶ class_names = sorted(train_csv["label"].unique())

label_to_index = {
    label: index
    for index, label in enumerate(class_names)
}

index_to_label = {
    index: label
    for label, index in label_to_index.items()
}
```

```
print("Number of classes:", len(class_names))
print(class_names[:10])
```

```
Number of classes: 75
['ADONIS', 'AFRICAN GIANT SWALLOWTAIL', 'AMERICAN SNOOT', 'AN 88', 'APOLLO', 'ATALA', 'BANDED ORANGE HELICONIAN', 'BANDED PEACOCK', 'BECKERS WHITE', 'BLACK HAIRSTREAK']
```

```
train_df = train_df.copy()
val_df = val_df.copy()
```

```
train_df["label_id"] = train_df["label"].map(label_to_index)
val_df["label_id"] = val_df["label"].map(label_to_index)
```

```
train_df["filepath"] = train_df["filename"].apply(
    lambda x: os.path.join(
        "/content/butterfly_dataset/train",
        x
    )
)

val_df["filepath"] = val_df["filename"].apply(
    lambda x: os.path.join(
        "/content/butterfly_dataset/train",
        x
    )
)
```

```
train_df.head()
```

	filename	label	label_id	filepath
4490	Image_4491.jpg	MANGROVE SKIPPER	40	/content/butterfly_dataset/train/Image_4491.jpg
4800	Image_4801.jpg	GREAT JAY	32	/content/butterfly_dataset/train/Image_4801.jpg
397	Image_398.jpg	SLEEPY ORANGE	64	/content/butterfly_dataset/train/Image_398.jpg
6140	Image_6141.jpg	CLEOPATRA	17	/content/butterfly_dataset/train/Image_6141.jpg
973	Image_974.jpg	PAINTED LADY	49	/content/butterfly_dataset/train/Image_974.jpg

Next steps: [Generate code with train_df](#) [New interactive sheet](#)

```
import os
```

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

```
import tensorflow as tf
```

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
```

```
def test_load_image(filepath, label):
    image = tf.io.read_file(filepath)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.cast(image, tf.float32) / 255.0
    return image, label
```

```
print("Function created successfully!")
print(test_load_image)
```

```
Function created successfully!
<function test_load_image at 0x787f82e48f40>
```

```
sample_path = train_df["filepath"].iloc[0]
sample_label = train_df["label_id"].iloc[0]

image, label = test_load_image(sample_path, sample_label)

print("Image shape:", image.shape)
print("Label:", label)

print("Minimum pixel value:", tf.reduce_min(image).numpy())
print("Maximum pixel value:", tf.reduce_max(image).numpy())
```

```
Image shape: (224, 224, 3)
Label: 40
Minimum pixel value: 0.0
Maximum pixel value: 1.0
```

```
➤ train_ds = tf.data.Dataset.from_tensor_slices(
    (
        train_df["filepath"].values,
        train_df["label_id"].values
    )
)

val_ds = tf.data.Dataset.from_tensor_slices(
    (
        val_df["filepath"].values,
        val_df["label_id"].values
    )
)

train_ds = train_ds.shuffle(1000).batch(BATCH_SIZE).prefetch(
    tf.data.AUTOTUNE
)

val_ds = val_ds.batch(BATCH_SIZE).prefetch(
    tf.data.AUTOTUNE
)
```

```
▶ images, labels = next(iter(train_ds))

print("Image batch shape:", images.shape)
print("Label batch shape:", labels.shape)
print("Pixel minimum:", tf.reduce_min(images).numpy())
print("Pixel maximum:", tf.reduce_max(images).numpy())
```

```
... Image batch shape: (32, 224, 224, 3)
Label batch shape: (32,)
Pixel minimum: 0.0
Pixel maximum: 1.0
```

```
from tensorflow.keras import layers, models

model = models.Sequential([

    # Input
    layers.Input(shape=(224, 224, 3)),

    # Convolution Block 1
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Convolution Block 2
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Convolution Block 3
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Classification layers
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    # Output layer
    layers.Dense(75, activation='softmax')

])
```

▶ `model.summary()`

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 75)	9,675

Total params: 11,178,635 (42.64 MB)
Trainable params: 11,178,635 (42.64 MB)
Non-trainable params: 0 (0.00 B)

```
images, labels = next(iter(train_ds))

predictions = model(images)

print("Input shape:", images.shape)
print("Prediction shape:", predictions.shape)
```

Input shape: (32, 224, 224, 3)
Prediction shape: (32, 75)

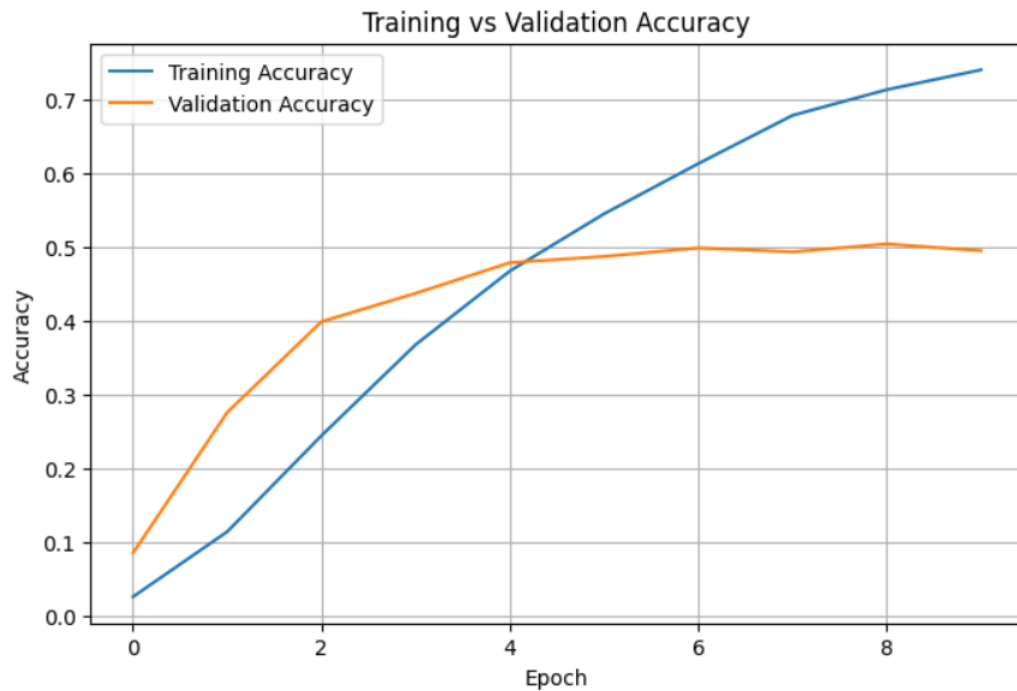
```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=10  
)
```

```
... Epoch 1/10  
163/163 ————— 680s 4s/step - accuracy: 0.0262 - loss: 4.2709 - val_accuracy: 0.0854 - val_loss: 3.8947  
Epoch 2/10  
163/163 ————— 676s 4s/step - accuracy: 0.1144 - loss: 3.6525 - val_accuracy: 0.2762 - val_loss: 3.0045  
Epoch 3/10  
163/163 ————— 643s 4s/step - accuracy: 0.2447 - loss: 2.9442 - val_accuracy: 0.3992 - val_loss: 2.4501  
Epoch 4/10  
163/163 ————— 639s 4s/step - accuracy: 0.3681 - loss: 2.3589 - val_accuracy: 0.4377 - val_loss: 2.1974  
Epoch 5/10  
163/163 ————— 677s 4s/step - accuracy: 0.4680 - loss: 1.9126 - val_accuracy: 0.4792 - val_loss: 2.0399  
Epoch 6/10  
163/163 ————— 658s 4s/step - accuracy: 0.5455 - loss: 1.5714 - val_accuracy: 0.4877 - val_loss: 1.9803  
Epoch 7/10  
163/163 ————— 605s 4s/step - accuracy: 0.6136 - loss: 1.2726 - val_accuracy: 0.4992 - val_loss: 1.9355  
Epoch 8/10  
163/163 ————— 661s 4s/step - accuracy: 0.6790 - loss: 1.0812 - val_accuracy: 0.4938 - val_loss: 2.1350  
Epoch 9/10  
163/163 ————— 605s 4s/step - accuracy: 0.7138 - loss: 0.9296 - val_accuracy: 0.5046 - val_loss: 2.0394  
Epoch 10/10  
163/163 ————— 652s 4s/step - accuracy: 0.7407 - loss: 0.8274 - val_accuracy: 0.4954 - val_loss: 2.2051
```

```
plt.show()
```

...



```
best_epoch = history.history["val_accuracy"].index(
    max(history.history["val_accuracy"])
) + 1

best_val_accuracy = max(history.history["val_accuracy"])

print("Best epoch:", best_epoch)
print("Best validation accuracy:", best_val_accuracy)
```

```
Best epoch: 9
Best validation accuracy: 0.5046153664588928
```

```
val_loss, val_accuracy = model.evaluate(val_ds)

print("Validation Loss:", val_loss)
print("Validation Accuracy:", val_accuracy)
```

```
41/41 ————— 44s 1s/step - accuracy: 0.4954 - loss: 2.2051
Validation Loss: 2.2050681114196777
Validation Accuracy: 0.4953846037387848
```

▶ `import numpy as np`

```
y_true = []
y_pred = []

for images, labels in val_ds:
    predictions = model.predict(images, verbose=0)

    y_true.extend(labels.numpy())
    y_pred.extend(np.argmax(predictions, axis=1))

y_true = np.array(y_true)
y_pred = np.array(y_pred)

print("Number of true labels:", len(y_true))
print("Number of predictions:", len(y_pred))
```

```
... Number of true labels: 1300
    Number of predictions: 1300
```

	precision	recall	f1-score	support
...				
ADONIS	0.83	0.83	0.83	18
AFRICAN GIANT SWALLOWTAIL	0.88	0.47	0.61	15
AMERICAN SNOOT	0.18	0.13	0.15	15
AN 88	1.00	0.88	0.94	17
APPOLLO	0.34	0.78	0.47	18
ATALA	0.62	0.80	0.70	20
BANDED ORANGE HELICONIAN	0.54	0.70	0.61	20
BANDED PEACOCK	0.53	0.53	0.53	17
BECKERS WHITE	0.15	0.19	0.17	16
BLACK HAIRSTREAK	1.00	0.41	0.58	17
BLUE MORPHO	0.83	0.33	0.48	15
BLUE SPOTTED CROW	0.33	0.47	0.39	17
BROWN SIPROETA	0.48	0.50	0.49	20
CABBAGE WHITE	0.48	0.61	0.54	18
CAIRNS BIRDWING	0.58	0.41	0.48	17
CHECQUERED SKIPPER	0.52	0.74	0.61	19
CHESTNUT	0.38	0.35	0.36	17
CLEOPATRA	0.17	0.21	0.19	19
CLODIUS PARNASSIAN	0.22	0.12	0.15	17
CLOUDED SULPHUR	0.32	0.33	0.32	18
COMMON BANDED AWL	0.33	0.24	0.28	17
COMMON WOOD-NYMPH	0.17	0.22	0.20	18
COPPER TAIL	0.13	0.21	0.16	19
CRECENT	0.58	0.55	0.56	20
CRIMSON PATCH	0.65	0.79	0.71	14
DANAID EGGFLY	0.14	0.11	0.12	19
EASTERN COMA	0.46	0.58	0.51	19
EASTERN DAPPLE WHITE	0.50	0.33	0.40	18
EASTERN PINE ELFEN	0.71	0.53	0.61	19
PINE WHITE	0.43	0.35	0.39	17
PIPEVINE SWALLOW	0.82	0.53	0.64	17
POPINJAY	0.77	0.59	0.67	17
PURPLE HAIRSTREAK	0.23	0.19	0.21	16
PURPLISH COPPER	0.27	0.17	0.21	18
QUESTION MARK	0.70	0.47	0.56	15
RED ADMIRAL	0.52	0.75	0.62	16
RED CRACKER	0.52	0.68	0.59	19
RED POSTMAN	0.77	0.56	0.65	18
RED SPOTTED PURPLE	0.44	0.65	0.52	17
SCARCE SWALLOW	0.82	0.90	0.86	20
SILVER SPOT SKIPPER	0.53	0.53	0.53	17
SLEEPY ORANGE	0.56	0.23	0.32	22
SOOTYWING	0.56	0.28	0.37	18
SOUTHERN DOGFACE	0.33	0.35	0.34	17
STRAITED QUEEN	0.92	0.65	0.76	17
TROPICAL LEAFWING	0.25	0.06	0.10	17
TWO BARRED FLASHER	0.58	0.47	0.52	15
ULYSES	0.84	0.94	0.89	17
VICEROY	0.44	0.69	0.54	16
WOOD SATYR	0.42	0.36	0.38	14
YELLOW SWALLOW TAIL	0.59	0.67	0.62	15
ZEBRA LONG WING	0.63	0.80	0.71	15
accuracy			0.50	1300
macro avg	0.52	0.49	0.49	1300
weighted avg	0.52	0.50	0.49	1300

